

// Jivops

Ceph

Guía completa — 30 lecciones

Fundamentos · Datos y replicación · Servicios RBD/CephFS/RGW ·
Integración · Rendimiento y operación · Producción avanzada

Contenido

Módulo 1: Fundamentos de Ceph

- Lección 01 — Qué es Ceph y qué problema resuelve
- Lección 02 — Arquitectura de Ceph — RADOS, OSD, MON y MGR
- Lección 03 — CRUSH — colocar datos sin servidor central de ubicaciones
- Lección 04 — Pools y placement groups
- Lección 05 — Instalación y operación con cephadm

Módulo 2: Datos y replicación

- Lección 06 — Replicación frente a erasure coding
- Lección 07 — Dominios de fallo y reglas CRUSH
- Lección 08 — Estados de salud del cluster y de los PG
- Lección 09 — Recuperación y backfill tras un fallo
- Lección 10 — Scrubbing y deep scrubbing

Módulo 3: Los servicios de Ceph — RBD, CephFS y RGW

- Lección 11 — RBD — almacenamiento en bloque
- Lección 12 — CephFS y los servidores de metadatos
- Lección 13 — RGW — almacenamiento de objetos compatible con S3
- Lección 14 — Snapshots y clones en RBD
- Lección 15 — RGW multisite — replicación entre zonas

Módulo 4: Integración y seguridad

- Lección 16 — Ceph en Kubernetes — Rook y ceph-csi
- Lección 17 — RBD frente a CephFS en Kubernetes — modos de acceso
- Lección 18 — Ceph como almacenamiento de máquinas virtuales
- Lección 19 — Autenticación con cephx y capabilities
- Lección 20 — Cifrado en reposo y en tránsito

Módulo 5: Rendimiento y operación

- Lección 21 — BlueStore, WAL y DB en dispositivos rápidos
- Lección 22 — Redes pública y de cluster
- Lección 23 — Autoscaler de PG y balancer
- Lección 24 — Añadir y retirar OSD y nodos
- Lección 25 — Actualizaciones rolling con cephadm

Módulo 6: Producción avanzada

Lección 26 — Monitoreo con Prometheus y el dashboard de Ceph

Lección 27 — Troubleshooting — OSD caídos y slow ops

Lección 28 — Capacidad y umbrales nearfull, backfillfull y full

Lección 29 — Backup y recuperación ante desastres

Lección 30 — Proyecto final: cluster Ceph listo para producción

Módulo 1: Fundamentos de Ceph

Lección 01 — Qué es Ceph y qué problema resuelve

Es una plataforma de almacenamiento definido por software que agrupa los discos de muchos servidores en un único cluster y ofrece almacenamiento de objetos, bloques y archivos sobre la misma capa base, creciendo al añadir nodos.

Un único cluster y un único conjunto de discos atienden cargas distintas (máquinas virtuales, archivos compartidos, objetos), así que se opera y monitoriza una sola plataforma en lugar de varias soluciones separadas.

Lección 02 — Arquitectura de Ceph — RADOS, OSD, MON y MGR

Los OSD almacenan los datos y gestionan replicación y recuperación, normalmente uno por disco; los MON mantienen el mapa del cluster y su quórum; los MGR ofrecen métricas, dashboard y módulos de gestión.

Sin un servidor central en el camino de datos no hay cuello de botella ni punto único de fallo, y el rendimiento crece al añadir OSD porque cada cliente trabaja en paralelo con muchos de ellos.

Lección 03 — CRUSH — colocar datos sin servidor central de ubicaciones

Es un algoritmo que calcula, a partir del nombre del objeto y del mapa del cluster, en qué OSD debe estar cada dato, de modo que clientes y OSD lo averiguan sin consultar una tabla central.

Una tabla central crecería con el número de objetos y sería cuello de botella y punto de fallo; el cálculo determinista lo hace cada cliente con el mapa del cluster, sin carga extra por cada objeto nuevo.

Lección 04 — Pools y placement groups

Un pool agrupa datos con las mismas reglas (replicación, regla CRUSH); sus objetos se asignan a placement groups y CRUSH reparte esos PG entre los OSD, de modo que no se gestiona cada objeto por separado.

Hay muchísimos menos PG que objetos, así que replicación y recuperación se gestionan por grupo y no objeto a objeto, aunque el pool tenga millones de ellos.

Lección 05 — Instalación y operación con cephadm

Despliega los demonios como contenedores a través del orquestador: con cephadm bootstrap se crea el primer MON y MGR en un nodo, y luego se añaden hosts y servicios con los comandos ceph orch.

El orquestador parte del estado deseado y se encarga de desplegar y colocar los demonios; añadir un host o un disco pasa a ser un comando, y las actualizaciones se coordinan desde un único punto.

Módulo 2: Datos y replicación

Lección 06 — Replicación frente a erasure coding

La replicación guarda copias completas (por ejemplo 3), con baja latencia y recuperación sencilla pero triple de espacio; el erasure coding $k+m$ reparte el dato en fragmentos con paridad y ahorra espacio (1,5 veces con 4+2) a cambio de más CPU y latencia.

Cada escritura exige calcular paridad y repartir fragmentos entre varios OSD, y actualizar una parte pequeña obliga a leer y reescribir varios fragmentos; la replicación solo copia el dato.

Lección 07 — Dominios de fallo y reglas CRUSH

Define a qué nivel (host, rack, sala) deben separarse las réplicas: con dominio host, las copias de un PG nunca quedan en el mismo servidor, así que la caída de un host no elimina todas las copias.

Separar por rack protege frente a la caída de un rack entero (energía o switch), pero hacen falta al menos tantos racks como copias; sin suficientes dominios distintos, los PG no pueden colocarse como pide la regla.

Lección 08 — Estados de salud del cluster y de los PG

HEALTH_OK indica que no hay problemas; HEALTH_WARN avisa de condiciones que requieren atención aunque los datos siguen accesibles (por ejemplo un OSD caído con réplicas suficientes); HEALTH_ERR señala un problema grave, como PG no disponibles o cluster lleno.

El aviso suele indicar pérdida de redundancia; los datos siguen accesibles, pero un segundo fallo podría provocar pérdida de disponibilidad o de datos, y ignorarlo deja que los problemas se acumulen.

Lección 09 — Recuperación y backfill tras un fallo

Si un OSD estuvo caído poco tiempo se recupera reproduciendo los cambios que se perdió (recovery); si es nuevo o estuvo mucho tiempo fuera se copian los objetos completos (backfill); ambos consumen red y disco y compiten con el tráfico de los clientes.

La recuperación usa red y disco que también necesitan los clientes; un límite equilibra el rendimiento de las aplicaciones con la duración de la ventana de riesgo mientras faltan copias.

Lección 10 — Scrubbing y deep scrubbing

El scrub compara con frecuencia (por defecto a diario) los metadatos de las réplicas, y el deep scrub lee y verifica los datos con checksums (por defecto semanal), detectando corrupción silenciosa que no se notaría hasta leer ese dato.

Un dato corrupto puede pasar meses sin detectarse si nadie lo lee, y si después falla otra copia podría perderse también la buena; el scrub permite reparar la copia mala a partir de las buenas mientras existen.

Módulo 3: Los servicios de Ceph — RBD, CephFS y RGW

Lección 11 — RBD — almacenamiento en bloque

Imágenes de bloque con aprovisionamiento ligero, repartidas en objetos de un pool y accesibles mediante librbd o el módulo del kernel, apropiadas para discos de máquinas virtuales y volúmenes de bases de datos.

Las lecturas y escrituras se reparten en paralelo entre muchos OSD, así que el rendimiento no queda limitado por un solo disco, y la imagen hereda la replicación y distribución del pool.

Lección 12 — CephFS y los servidores de metadatos

Es un sistema de archivos POSIX distribuido: los datos viven en un pool de datos y la jerarquía de directorios y permisos la gestionan los servidores de metadatos (MDS), que pueden tener instancias activas y en espera.

Los clientes leen y escriben el contenido directamente en los OSD y solo consultan al MDS por la estructura de directorios, así que el tráfico de datos no pasa por el servidor de metadatos.

Lección 13 — RGW — almacenamiento de objetos compatible con S3

Una pasarela REST que ofrece almacenamiento de objetos compatible con S3 y Swift sobre RADOS, con usuarios, buckets y políticas, accesible por HTTP desde aplicaciones y herramientas estándar.

Las aplicaciones y herramientas que ya hablan S3 pueden apuntar al nuevo endpoint con pocos cambios, sin reescribir el acceso a datos ni depender de un proveedor concreto.

Lección 14 — Snapshots y clones en RBD

Un snapshot es una copia de solo lectura del estado de una imagen en un instante; un clone crea una imagen nueva a partir de un snapshot con copy-on-write, así que solo ocupa espacio por lo que cambia respecto al original.

El snapshot reside en el mismo cluster y se pierde si el cluster o el pool se pierden; protege frente a errores lógicos, pero una copia de seguridad debe estar separada del sistema original.

Lección 15 — RGW multisite — replicación entre zonas

Organiza realm, zonegroups y zonas, y replica datos y metadatos de forma asíncrona entre zonas de clusters distintos, útil para tener copias en otra ubicación o servir a usuarios cercanos, con consistencia eventual.

La escritura se confirma localmente antes de llegar a la otra zona; si el sitio principal cae antes de replicar, esos cambios aún no existen en el secundario, el precio de no esperar a la latencia entre ubicaciones.

Módulo 4: Integración y seguridad

Lección 16 — Ceph en Kubernetes — Rook y ceph-csi

Rook es un operador que despliega y gestiona Ceph dentro de Kubernetes mediante recursos personalizados, y ceph-csi es el driver CSI que permite a los pods crear y montar volúmenes RBD o CephFS a través de PVC.

Cada PVC genera automáticamente el volumen en Ceph sin que un administrador lo cree antes; los desarrolladores piden almacenamiento con un manifiesto y se evitan el trabajo manual y los errores de asociar volúmenes uno a uno.

Lección 17 — RBD frente a CephFS en Kubernetes — modos de acceso

Un volumen RBD con sistema de archivos se usa normalmente con ReadWriteOnce (un nodo a la vez), mientras CephFS permite ReadWriteMany, es decir, que varios pods en nodos distintos lo monten a la vez.

Ext4 no coordina el acceso concurrente desde varios nodos; cada uno tendría su propia caché y su visión de los metadatos, y las escrituras simultáneas corromperían el sistema de archivos. Para acceso compartido hace falta un sistema distribuido como CephFS.

Lección 18 — Ceph como almacenamiento de máquinas virtuales

Los discos viven en el cluster y no en el hipervisor, así que una VM puede arrancar en otro host tras un fallo o migrarse en caliente sin copiar su disco.

Si un hipervisor falla, otro puede arrancar la VM porque su disco es accesible desde cualquier host; no hay que restaurar ni copiar nada, ya que el dato no depende del servidor que ejecuta la máquina.

Lección 19 — Autenticación con cephx y capabilities

Autentica a clientes y demonios con claves guardadas en un keyring, y cada usuario recibe capabilities que limitan lo que puede hacer (por ejemplo allow rw sobre un pool concreto), aplicando el principio de menor privilegio.

Si la aplicación se compromete, el atacante solo alcanza ese pool y no todo el cluster; client.admin tiene control total y su filtración sería crítica, así que los permisos mínimos reducen el impacto de un error o una intrusión.

Lección 20 — Cifrado en reposo y en tránsito

El cifrado en reposo se activa por OSD con dm-crypt (LUKS) y protege los datos si se roba un disco; el cifrado en tránsito usa el modo secure del protocolo msgr2 entre clientes y demonios; son protecciones independientes.

El cifrado en reposo actúa sobre el contenido del disco; el tráfico de red viaja sin cifrar si no se activa también el cifrado en tránsito. Cubren amenazas distintas (robo de disco frente a captura de red), así que hay que habilitar ambos.

Módulo 5: Rendimiento y operación

Lección 21 — BlueStore, WAL y DB en dispositivos rápidos

Escribe directamente en el dispositivo de bloque sin sistema de archivos intermedio, guarda metadatos en RocksDB y verifica checksums; colocar la DB y el WAL en SSD o NVMe acelera sobre todo los clusters con discos HDD.

Los metadatos generan muchas operaciones pequeñas y aleatorias, justo lo que peor hace un HDD; el SSD las atiende con baja latencia y deja al disco mecánico los datos secuenciales.

Lección 22 — Redes pública y de cluster

La red pública conecta clientes y demonios; la red de cluster, opcional, se dedica al tráfico entre OSD (replicación, recuperación y latidos), de modo que una recuperación intensa no compita con el tráfico de los clientes.

Cada escritura del cliente se multiplica entre los OSD que guardan las réplicas; con tres copias el tráfico interno supera al del cliente, y la recuperación tras un fallo añade aún más, así que la red puede ser el cuello de botella.

Lección 23 — Autoscaler de PG y balancer

El autoscaler ajusta el número de PG de cada pool según su uso y objetivo previsto, evitando pocos PG (reparto desigual) o demasiados (sobrecarga de memoria y CPU); el balancer redistribuye PG para igualar el uso de los OSD.

Un OSD con más PG recibe más datos y alcanza antes los umbrales nearfull y full; el cluster se limita por el OSD más lleno y no por la media, así que ese OSD puede bloquear escrituras aunque quede mucho espacio en total.

Lección 24 — Añadir y retirar OSD y nodos

Al añadir un OSD, CRUSH reasigna parte de los PG hacia él y se hace backfill; para retirarlo se marca out, se espera a que los datos migren y el cluster vuelva a active+clean, y solo entonces se elimina.

Si el disco desaparece de golpe, sus copias dejan de existir y el cluster queda con menos redundancia hasta recuperarlas; marcarlo out primero permite copiar los datos mientras el disco aún sirve de origen, acortando la ventana de riesgo.

Lección 25 — Actualizaciones rolling con cephadm

Actualiza los demonios de forma escalonada y en un orden seguro (MGR, MON, OSD...), comprobando la salud del cluster entre pasos y permitiendo pausar o detener la actualización.

Actualizar reinicia demonios, y con menos redundancia un reinicio adicional podría dejar datos no disponibles; además, un problema previo se mezclaría con los efectos de la actualización y sería difícil de diagnosticar.

Módulo 6: Producción avanzada

Lección 26 — Monitoreo con Prometheus y el dashboard de Ceph

El módulo prometheus del MGR expone métricas del cluster (salud, uso por OSD y por pool, latencias) que recoge Prometheus y se visualizan en Grafana, con alertas sobre salud, capacidad y OSD caídos.

El OSD más lleno determina cuándo se bloquean las escrituras, no el promedio; un reparto desigual puede esconder un OSD cerca de nearfull aunque el uso global sea bajo, y las métricas por OSD permiten actuar a tiempo.

Lección 27 — Troubleshooting — OSD caídos y slow ops

Empezar por ceph -s y ceph health detail, identificar qué PG y OSD están afectados, revisar logs y métricas de disco y red del nodo, y distinguir si es un problema de hardware, de red o de saturación antes de actuar.

Reiniciar todo a la vez deja el cluster sin redundancia y desencadena una recuperación masiva, pierde el estado que ayudaría a diagnosticar y no ataca el origen real, que puede ser un disco o una red concretos.

Lección 28 — Capacidad y umbrales nearfull, backfillfull y full

Por defecto avisa en nearfull (85 %), deja de aceptar backfill en backfillfull (90 %) y bloquea las escrituras en full (95 %); como se evalúa por OSD, el más lleno manda, así que hay que ampliar antes de acercarse.

Cuando falla un OSD o un host, sus datos se recrean en los demás y necesitan espacio libre; sin margen la recuperación puede toparse con los umbrales y no completarse, y el reparto desigual hace que algunos OSD lleguen antes al límite.

Lección 29 — Backup y recuperación ante desastres

Protege frente a fallos de discos o nodos, pero no frente a borrados, corrupción lógica o pérdida del sitio; para recuperación ante desastres se usan RBD mirroring (asíncrono entre clusters), multisite en RGW y copias en otro lugar, definiendo RPO y RTO.

La replicación copia fielmente cualquier cambio, incluidos los borrados y la corrupción lógica; las copias son iguales entre sí, así que no hay una versión buena a la que volver y hace falta un snapshot o una copia de seguridad independiente.

Lección 30 — Proyecto final: cluster Ceph listo para producción

Al menos 3 MON y varios MGR, OSD repartidos por dominios de fallo con replicación o EC según la carga, redes pública y de cluster separadas, cephx con permisos mínimos y cifrado, monitoreo con alertas de capacidad y salud, y recuperación ante desastres con mirroring o multisite.

Intenta mencionar al menos dos pilares: monitores en número impar y dominios de fallo, redes separadas, cephx con permisos mínimos, monitoreo de capacidad o recuperación ante desastres.